
SQL - Funktionen

BITLC | Tom Selig
28. August 2026

Funktionen

- SQL Server bringt eine Reihe von Funktionen mit, die an fast beliebiger Stelle in SQL-Anweisungen verwendet werden können.
- Dabei handelt es sich um skalare Funktionen, also Funktionen die nur einen Wert als Ergebnis zurückliefern.
- Sie werden oft in der **SELECT**-Klausel eingesetzt, um Spalten im Ergebnis zu formatieren oder in der **WHERE**-Klausel, um Bedingungen zu formulieren.
- Funktionsaufrufe können ineinander verschachtelt werden.
- Funktionen stehen zu verschiedenen Themen bereit.

Logische Funktionen

- **IIF**(*<Logischer ausdruck>*, *<wenn true>*, *<wenn false>*)

Verkürzte Form einer IF-ELSE-Verzweigung, in der Programmierung als ternärer Operator bekannt.

Wenn *<logischer ausdruck>* true ergibt, wird *<wenn true>* zurückgeliefert, bei false wird *<wenn false>* zurückgeliefert.

- **ISNULL**(*<ausdruck>*, *<ersatz>*)

Liefert *<ersatz>*, wenn *<ausdruck>* NULL ist, ansonsten wird *<ausdruck>* zurückgeliefert.

- **COALESCE**(*<ausdruck1>*, *<ausdruck2>* [, ...*n*])

Liefert den Wert des ersten Ausdrucks der Liste zurück, der nicht NULL ist.

CASE-Ausdruck

- Der einfache **CASE**-Ausdruck überprüft einen gegebenen Wert auf Übereinstimmung mit verschiedenen anderen Werten:

```
CASE <wert>
  WHEN <vergleichswert1> THEN <ergebnis1>
  WHEN <vergleichswert2> THEN <ergebnis2>
  ...
  ELSE <ergebnis_sonst>
END
```

- Sobald der erste passende Vergleichswert gefunden wurde, wird das zugehörige Ergebnis geliefert und die Verarbeitung beendet. Alle folgenden Vergleichswerte werden nicht mehr berücksichtigt.
- Der **ELSE**-Zweig ist optional, ohne **ELSE** liefert **CASE** ggf. **NULL**.

CASE-Ausdruck

- Im Gegensatz zum einfachen **CASE**-Ausdruck kann der komplexe **CASE**-Ausdruck beliebige logische Ausdrücke verarbeiten:

```
CASE
    WHEN <ausdruck1> THEN <ergebnis1>
    WHEN <ausdruck2> THEN <ergebnis2>
    ...
    ELSE <ergebnis_sonst>
END
```

- Sobald der erste Ausdruck in der Liste true ergibt, wird das zugehörige Ergebnis geliefert und die Verarbeitung beendet.
- Beim komplexen **CASE** ist die Reihenfolge der Ausdrücke wichtig!

Datum-/Uhrzeit-Funktionen

- **GETDATE()**, **SYSDATETIME()**

Liefert den aktuellen Datum-/Zeit-Stempel. **GETDATE()** liefert einen datetime-Wert, **SYSDATETIME()** einen datetime2-Wert.

- **DAY(<datum>)**

Liefert den Tag des angegebenen Datums als Zahl.

- **MONTH(<datum>)**

Liefert den Monat des angegebenen Datums als Zahl.

- **YEAR(<datum>)**

Liefert das Jahr des angegebenen Datums als Zahl.

Datum-/Uhrzeit-Funktionen

- **DATEPART**(*<einheit>*, *<datum>*)

Liefert einen beliebigen Bestandteil des angegebenen Datums als Zahl.

Mögliche Werte
für *<einheit>*
sind:

Diese Einheiten
gelten auch für
weitere Datum-/
Zeit-Funktionen.

Einheit	Abkürzung	Beschreibung	Bereich
year	yy, yyyy	Jahr	1 - 9999
quarter	qq, q	Quartal	1 - 4
month	mm, m	Monat	1 - 12
dayofyear	dy, y	Tag des Jahres	1 - 366
day	dd, d	Tag des Monats	1 - 31
week	wk, ww	Kalenderwoche	1 - 53
iso_week	isowk, isoww	Kalenderwoche europäisch	1 - 53
weekday	dw, w	Wochentag	1 - 7
hour	hh	Stunde	0 - 23
minute	mi, n	Minute	0 - 59
second	ss, s	Sekunde	0 - 59

Datum-/Uhrzeit-Funktionen

- **DATENAME**(*<einheit>*, *<datum>*)

Macht im Prinzip das gleiche wie **DATEPART**(), gibt allerdings wenn möglich Zeichenketten (z.B. Montag, Januar, ...) zurück, sonst auch Zahlen.

- **DATEADD**(*<einheit>*, *<anzahl>*, *<datum>*)

Addiert die angegebene Anzahl der gewünschten Einheit (siehe vorherige Folie) zum angegebenen Datum hinzu (auch negativ).

- **DATEDIFF**(*<einheit>*, *<startdatum>*, *<enddatum>*)

Berechnet die Differenz zwischen dem Start- und dem Enddatum in der gewünschten Einheit (siehe vorherige Folie).

Datum-/Uhrzeit-Funktionen

- **DATEFROMPARTS**(*<jahr>*, *<monat>*, *<tag>*),
TIMEFROMPARTS(*<std>*, *<min>*, *<sek>*, *<brucht.>*, *<anz>*),
- **DATETIME2FROMPARTS**(*<jahr>*, *<monat>*, *<tag>*, *<std>*,
<min>, *<sek>*, *<brucht.>*, *<präz.>*)

Generiert einen Datum-, einen Zeit- bzw. eine Datum/Zeit-Wert aus den übergebenen Argumenten als date-, time- oder datetime2-Datentyp.

- **EOMONTH**(*<datum>* [, *<offset>*])

Gibt den letzten Tag des Monats für das angegebene Datum zurück. Der optionale *<offset>*-Wert wird den Monaten hinzugerechnet:

```
SELECT EOMONTH('2023-09-06', 3);    => 2023-12-31
```

String-Funktionen

- **CONCAT**(*<string1>*, *<string2>* [, ... *n*])

Verkettet (konkateniert) beliebig viele Zeichenketten (Strings) miteinander. Strings, die **NULL** als Ergebnis liefern, werden übersprungen.

- **CONCAT_WS**(*<tz>*, *<string1>*, *<string2>* [, ... *n*])

Funktioniert prinzipiell wie **CONCAT**(), als erstes Argument wird aber ein Trennzeichen angegeben, das jeweils zwischen die Strings gesetzt wird.

- **LEN**(*<string>*)

Liefert die Länge des Strings in Zeichen, **ohne** nachfolgende Leerzeichen.

- **DATALength**(*<ausdruck>*)

Liefert die Länge in Bytes, die der Ausdruck verbraucht.

String-Funktionen

- **LTRIM**(*<string>*)

RTRIM(*<string>*)

TRIM(*<string>*)

Entfernt Leerzeichen an der linken, der rechten oder an beiden Seiten des Strings. Leerzeichen innerhalb des Strings bleiben immer erhalten.

- **UPPER**(*<string>*)

LOWER(*<string>*)

Wandelt alle Buchstaben des Strings in Groß- bzw. Kleinbuchstaben um. Ziffern und Sonderzeichen bleiben unverändert.

String-Funktionen

- **LEFT**(*<string>*, *<anzahl>*)

RIGHT(*<string>*, *<anzahl>*)

Liefert die gewünschte Anzahl Zeichen des Strings von links oder rechts.

- **SUBSTRING**(*<string>*, *<start>*, *<Länge>*)

Liefert einen Teil des Strings in gewünschter Länge ab dem Startpunkt.

- **CHARINDEX**(*<suchstring>*, *<zielstring>*)

PATINDEX(*<muster>*, *<string>*)

Liefert die Position, an der der Suchstring oder das Muster im Zielstring zum ersten Mal gefunden werden. Muster können wie bei **LIKE** gebildet werden.

String-Funktionen

- **REPLACE**(*<string>*, *<suchstring>*, *<ersatzstring>*)
Ersetzt in String alle Vorkommen des Suchstrings durch den Ersatzstring.
- **REVERSE**(*<string>*)
Dreht die Reihenfolge der Zeichen im String um.
- **ISNUMERIC**(*<ausdruck>*)
Prüft, ob sich *<ausdruck>* in einen gültigen numerischen Typ wandeln lässt.
- **SPACE**(*<anzahl>*)
Erzeugt eine Zeichenkette mit der gegebenen Anzahl von Leerzeichen.

Mathematische Funktionen

- **ABS**(*<wert>*)

Liefert den absoluten Wert einer Zahl, also ohne ggf. negatives Vorzeichen.

- **RAND**(*[<startwert>]*)

Liefert eine Zufallszahl zwischen 0 und 1. Bei Angabe eines konstanten Startwerts wird immer dieselbe Zahl geliefert.

- **ROUND**(*<wert>*, *<stelle>*, *[<funktion>]*)

Rundet den Wert auf die angegebene Anzahl Stellen. Wenn für Funktion ein Wert ungleich 0 angegeben ist, wird nicht gerundet sondern abgeschnitten.

- **POWER**(*<wert>*, *<potenz>*)

Liefert den potenzierten Wert

Konvertierung

- **CAST**(*<wert>* **AS** *<datentyp>*)

TRY_CAST(*<wert>* **AS** *<datentyp>*)

Wandelt den angegebenen Wert in den angegebenen Datentyp um. Ist dies nicht möglich, liefert **CAST**() einen Fehler, **TRY_CAST**() dagegen NULL.

- **CONVERT**(*<datentyp>*, *<wert>* [, *<format>*])

TRY_CONVERT(*<datentyp>*, *<wert>* [, *<format>*])

Wandelt den angegebenen Wert in den angegebenen Datentyp um. Ist dies nicht möglich, liefert **CONVERT**() einen Fehler und **TRY_CONVERT**() NULL.

Mit dem optionalen Parameter Format kann eine bestimmte Formatierung erzwungen werden, z. B. bei Datum-/Zeitangaben.

Formatierung

- **FORMAT**(*<wert>*, *<formatbezeichner>* [, *<kultur>*])

Formatiert den angegebenen Wert gemäß dem angegebenen Formatbezeichner und der ggf. angegebenen Kultur.

Wird für Datums-/Zeitangaben sowie für numerische Werte verwendet.

Der Formatbezeichner kann eine beliebige gültige Formatzeichenfolge aus dem .NET-Framework sein. Entweder als Standardformatzeichenfolge oder als Muster aus benutzerdefinierten Zeichen.

Als optionaler Parameter kann eine vom .NET-Framework unterstützte Kultur angegeben werden. Fehlt das Argument, wird die Kultur der aktuellen Sitzung verwendet.

```
SELECT FORMAT(123.4567, 'C', 'de-DE');    => 123,46 €
```

currency Deutsch